

12 Univariate Data Visualizations With Illustrations in Python

[Guest Blog](#)

Last Updated : 02 May, 2025



27

Univariate Data Visualization is a fundamental technique in data analysis that focuses on representing and interpreting a single variable at a time. This article explores the importance of data visualization, particularly for univariate data, and its role in uncovering meaningful insights. We'll delve into various methods and tools for visualizing both continuous and categorical univariate data using Python libraries such as Matplotlib and Seaborn. From simple scatter plots to more complex violin plots, we'll cover a range of techniques that help analysts and data scientists effectively communicate patterns, distributions, and trends within single-variable datasets.

Learning Objectives:

- Understand the purpose and importance of data visualization
- Learn various techniques for visualizing univariate continuous data
- Explore methods for visualizing categorical variables
- Gain practical skills in using Python libraries for data visualization

Table of contents

1. Need for Visualizing Data

2. Visualizing Univariate Continuous Data

- Univariate enumerative Plots
- Uni-variate summary plots :

3. VISUALIZING CATEGORICAL VARIABLES :

4. Conclusion

- References

Free Certification Courses



Analyzing Data with Power BI

Power BI Basics • Power Query • SQL • Building Dashboards • Analyze Business • Hands-on Projects

Get Certified Now

Need for Visualizing Data

- Understand the trends and patterns of data
- Analyze the frequency and other such characteristics of data
- Know the distribution of the variables in the data.
- Visualize the relationship that may exist between different variables

The number of variables of interest featured by the data classifies it as **univariate**, **bivariate**, or **multivariate**. For eg., If the data features only one variable of interest

then it is a uni-variate data. Further, based on the characteristics of data, it can be classified as **categorical/discrete** and **continuous** data.

In this article, the main focus is on **univariate analysis** data visualization(data is visualized in one-dimension). For the purpose of illustration, the '**iris**' data set is considered. The iris data set contains 3 classes of 50 instances each, where each class refers to a type of iris plant. The different variables involved in the data set are Sepal Length, Sepal Width, Petal Length, Petal width which is continuous and Variety which is a categorical variable. Though the data set is multivariate in nature, for univariate analysis in python, we consider one variable of interest at a time.

We proceed by first importing the required libraries and the data set. You can download the python notebook and dataset [here](#).

Importing libraries

```
In [1]: import pandas as pd
import numpy as np
```

Reading data using read_csv() function

```
In [2]: df=pd.read_csv('iris.csv')
```

Displaying the DataFrame

```
In [3]: df
```

Out[3]:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Virginica
146	6.3	2.5	5.0	1.9	Virginica
147	6.5	3.0	5.2	2.0	Virginica
148	6.2	3.4	5.4	2.3	Virginica
149	5.9	3.0	5.1	1.8	Virginica

150 rows × 5 columns

Python Code:

```
import pandas as pd
import numpy as np

df = pd.read_csv('iris.csv')
print(df)
```

Copy Code

The data set originally in .csv format is loaded into the DataFrame df using the `pd.read_csv()` function of pandas . Then, it displays the DataFrame df.

Before analyzing any data set, inspect the data types of the data variables. Then, one can decide on the right methods for univariate analysis data visualization.

Finding the data types of variables in the DataFrame

```
In [4]: df.dtypes
```

```
Out[4]: sepal.length    float64  
        sepal.width     float64  
        petal.length     float64  
        petal.width      float64  
        variety         object  
        dtype: object
```

The `.dtypes` property is used to know the data types of the variables in the data set. Pandas stores these variables in different formats according to their type. Pandas stores categorical variables as `'object'` and, on the other hand, continuous variables are stored as `int` or `float`. The methods used for visualization of univariate data also depends on the types of data variables.

In this article, we visualize the iris data using the libraries: `matplotlib` and `seaborn`. We use `Matplotlib` library to draw basic plots. `Seaborn` library is based on the `matplotlib` library and it provides a wide variety of visualization techniques for univariate analysis.

Importing libraries essential for data visualization

```
#MATPLOTLIB  
import matplotlib.pyplot as plt  
%matplotlib inline  
#SEABORN  
import seaborn as sns
```

Visualizing Univariate Continuous Data

Univariate data visualization plots help us comprehend the enumerative properties as well as a descriptive summary of the particular data variable. These plots help in

understanding the **location/position** of observations in the data variable, its **distribution**, and **dispersion**. Uni-variate plots are of two types: 1) Enumerative plots and 2) Summary plots

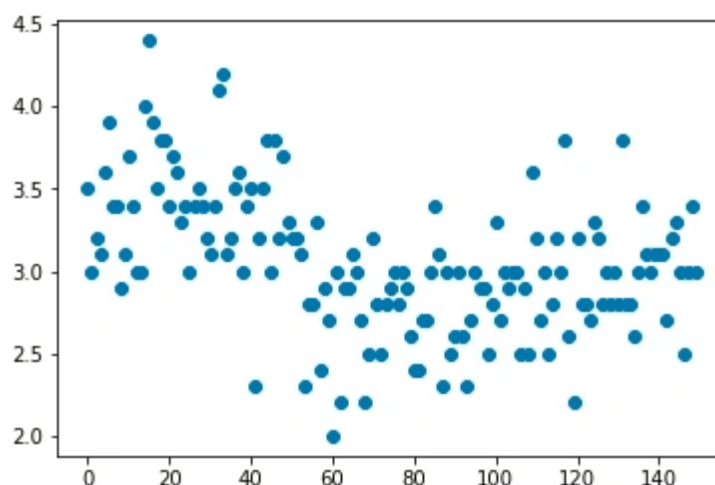
Univariate enumerative Plots

These plots enumerate/show every observation in data and provide information about the distribution of the observations on a single data variable. We now look at different enumerative plots.

1. UNIVARIATE SCATTER PLOT :

This plots different observations/values of the same variable corresponding to the index/observation number. Consider plotting of the variable 'sepal length(cm)' :

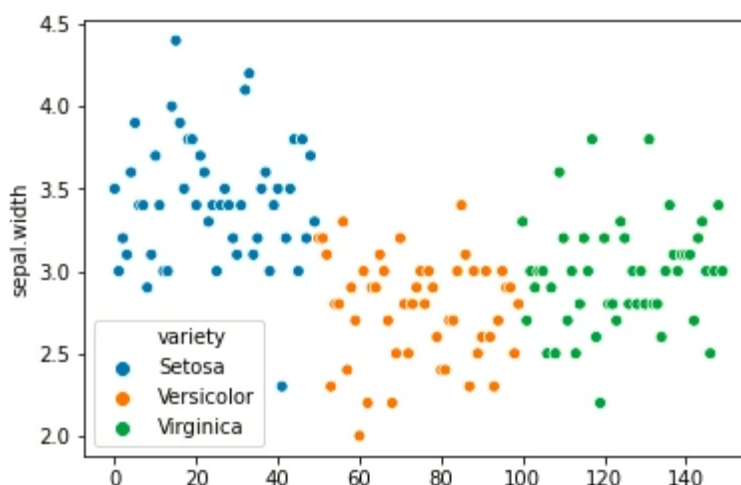
```
plt.scatter(df.index, df['sepal.width'])  
plt.show()
```



Use the `plt.scatter()` function of matplotlib to plot a univariate analysis scatter diagram. The `scatter()` function requires two parameters to plot. In this example, we plot the variable 'sepal.width' against the corresponding observation number stored as the index of the data frame (`df.index`).

Then visualize the same plot by considering its variety using the `sns.scatterplot()` function of the seaborn library.

```
sns.scatterplot(x=df.index, y=df['sepal.width'], hue=df['variety'])
```



In seaborn, the 'hue' parameter, an interesting feature, determines which column in the data frame to use for color encoding. It differentiates data values by their categories, assigning different colors based on the grouping variable provided as input.. The variable passed onto 'hue' can be either categorical or numeric, although color mapping will behave differently in the latter case.

“

Note: Every function has got a wide variety of parameters to play with to produce better results. If one is using Jupyter notebook, the various parameters of the function used can be explored by using the 'Shift+Tab' shortcut.

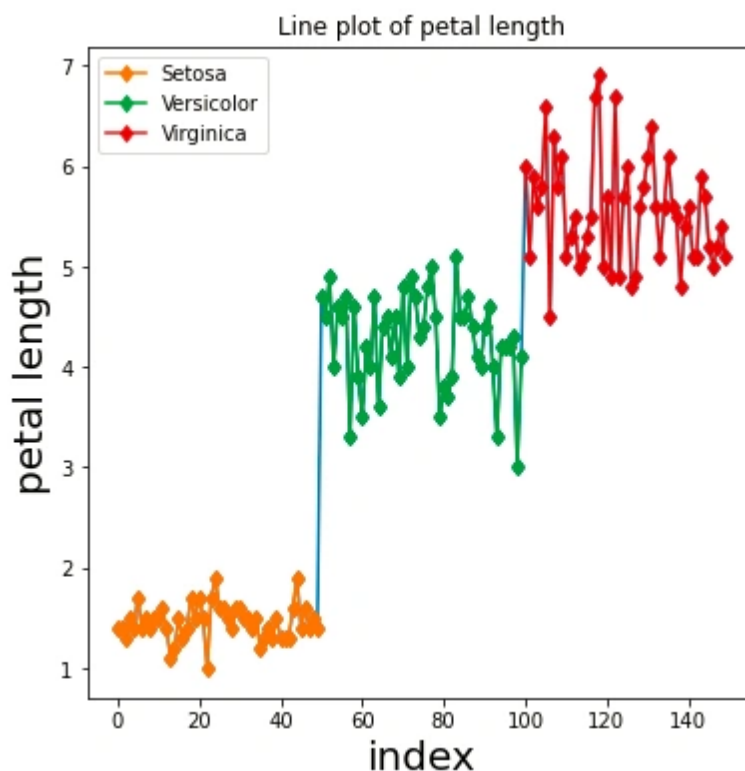
2. LINE PLOT (with markers) :

A line plot visualizes data by connecting the data points via line segments. It resembles a scatter plot but differs by ordering the measurement points (usually by

their x-axis value) and connecting them with straight line segments.

Setting title, figure size, labels and font size in matplotlib

```
plt.figure(figsize=(6,6))
plt.title('Line plot of petal length')
plt.xlabel('index',fontsize=20)
plt.ylabel('petal length',fontsize=20)
plt.plot(df.index,df['petal.length'],markevery=1,marker='d')
for name, group in df.groupby('variety'):
    plt.plot(group.index, group['petal.length'], label=name,markevery=1,marker='d')
plt.legend()
plt.show()
```



The matplotlib `plt.plot()` function by default plots the data using a line plot.

Previously, we discussed the `hue` parameter of seaborn. While matplotlib does not offer an automated option for this, one can utilize pandas' `groupby()` function to plot such a graph effectively.

Note: In the above illustration, we also implement methods for setting title, font size, etc., in matplotlib.

— Explanation of the functions used :

- `plt.figure(figsize=())` : To set the size of figure
- `plt.title()` : To set title
- `plt.xlabel()` / `plt.ylabel()` : To set labels on X-axis/Y-axis
- `df.groupby()` : To group the rows of the data frame according to the parameter passed onto the function
- The `groupby()` function returns the data frames grouped by the criterion variable passed and the criterion variable.
- The *for loop* is used to plot each data point according to its variety.
- `plt.legend()`: Adds a legend to the graph (Legend describes the different elements seen in the graph).
- `plt.show()` : to show the plot.

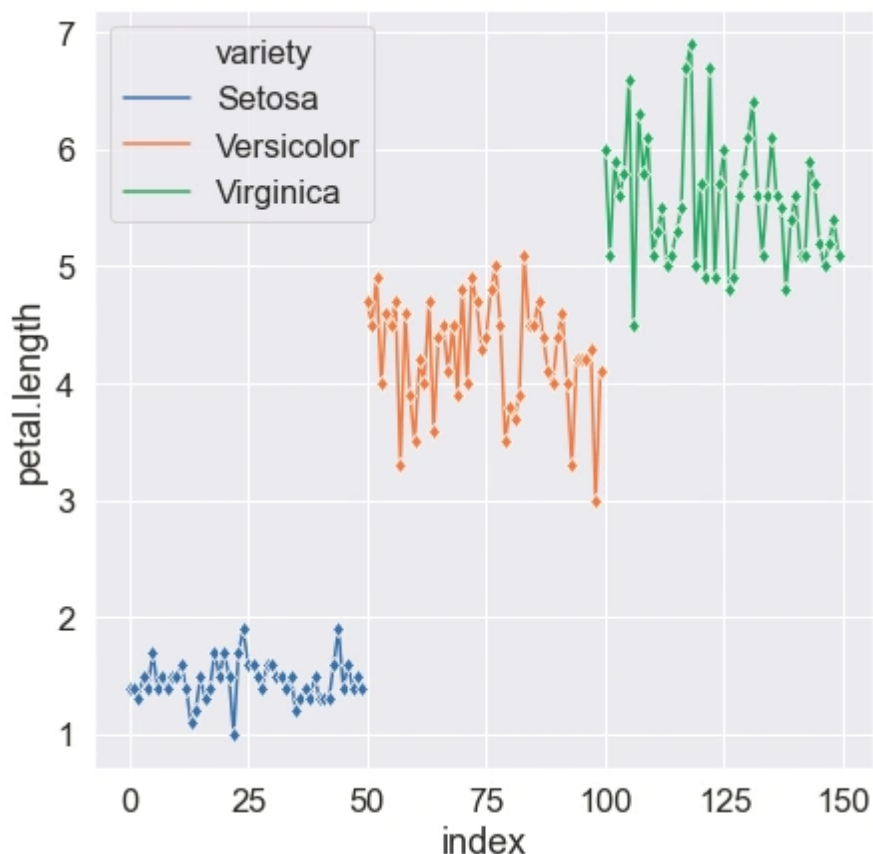
The 'markevery' parameter of the function `plt.plot()` is assigned to '1' which means it will plot every 1st marker starting from the first data point. There are various marker styles which we can pass as a parameter to the function.

The `sns.lineplot()` function can also visualize the line plot.

Setting title, figure size, labels and font size in seaborn

```
➤ sns.set(rc={'figure.figsize': (7,7)})  
sns.set(font_scale=1.5)
```

```
➤ fig=sns.lineplot(x=df.index,y=df['petal.length'],markevery=1,marker='d',data=df,hue=df['variety'])  
fig.set(xlabel='index')
```



In seaborn, the labels on axes are automatically set based on the columns that are passed for plotting. However if one desires to change it, it is possible too using the `set()` function.



Note: There are often cases wherein one would want to explore how the distribution of a single continuous variable is affected by a second categorical variable. The seaborn library provides a variety of plots that help perform such types of comparisons between uni-variate distributions. Three such plots are discussed in this article : Strip plot, Swarm plot (under enumerative plots), and Violin plot (under summary plots). The hue parameter mentioned in above plots is also for similar use.

3. STRIP PLOT :

The strip plot is similar to a scatter plot. It is often used along with other kinds of plots for better analysis. It is used to visualize the distribution of data points of the variable.

The `sns.stripplot()` function is used to plot a strip-plot :

```
sns.stripplot(y=df['sepal.width'])
```

It also helps to plot the distribution of variables for each category as individual data points. The function by default creates a vertical strip plot, plotting distributions of continuous data along the Y-axis and spacing categories along the X-axis without consideration in the above plot. Considering the categories helps in better visualization as seen in the below plot.

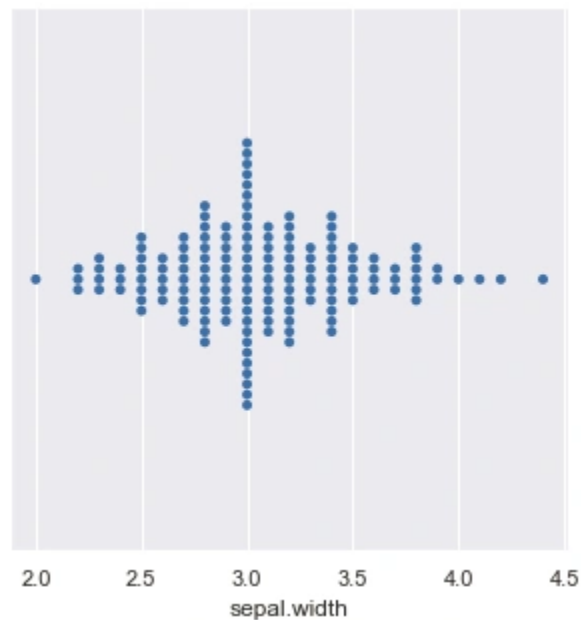
```
sns.stripplot(x=df['variety'], y=df['sepal.width'])
```

4. SWARM PLOT :

The swarm-plot, similar to a strip-plot, provides a visualization technique for univariate data to view the spread of values in a continuous variable. The only difference between the strip-plot and the swarm-plot is that the swarm-plot spreads out the data points of the variable automatically to avoid overlap and hence provides a better visual overview of the data.

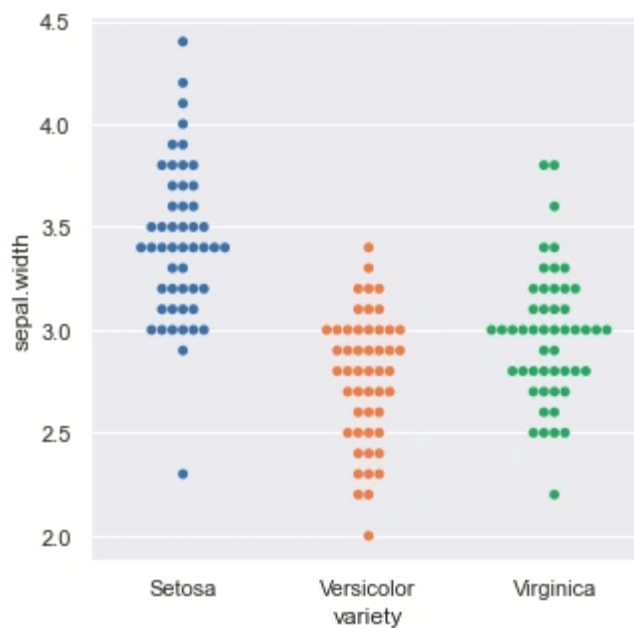
The `sns.swarmplot()` function is used to plot a swarm-plot :

```
sns.set(rc={'figure.figsize': (5,5)})  
sns.swarmplot(x=df['sepal.width'])
```



Distribution of the variable 'sepal.width' according to the categories :

```
sns.swarmplot(x=df['variety'], y=df['sepal.width'])
```



Uni-variate summary plots :

These plots give a more concise description of the location, dispersion, and distribution of a variable than an enumerative plot. Summarizing every individual data

value in a plot isn't feasible, but it efficiently represents the entire dataset, enabling better conclusions to be drawn.

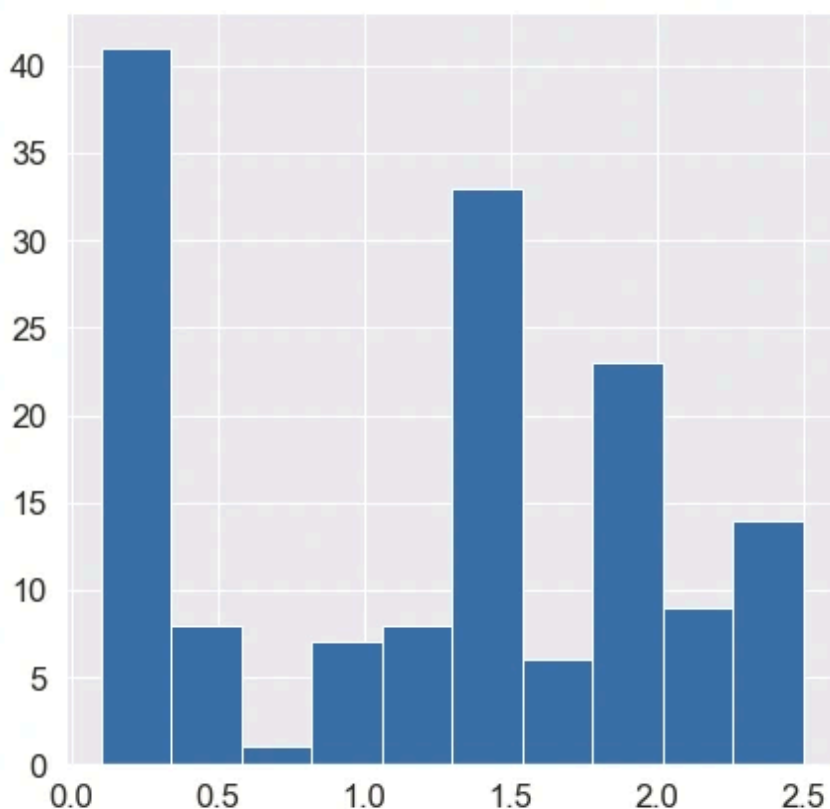
5. HISTOGRAMS :

Histograms are similar to bar charts which display the counts or relative frequencies of values falling in different class intervals or ranges. A histogram displays the shape and spread of continuous sample data. It also helps us understand the skewness and kurtosis of the distribution of the data.

Plotting histogram using the matplotlib *plt.hist()* function :

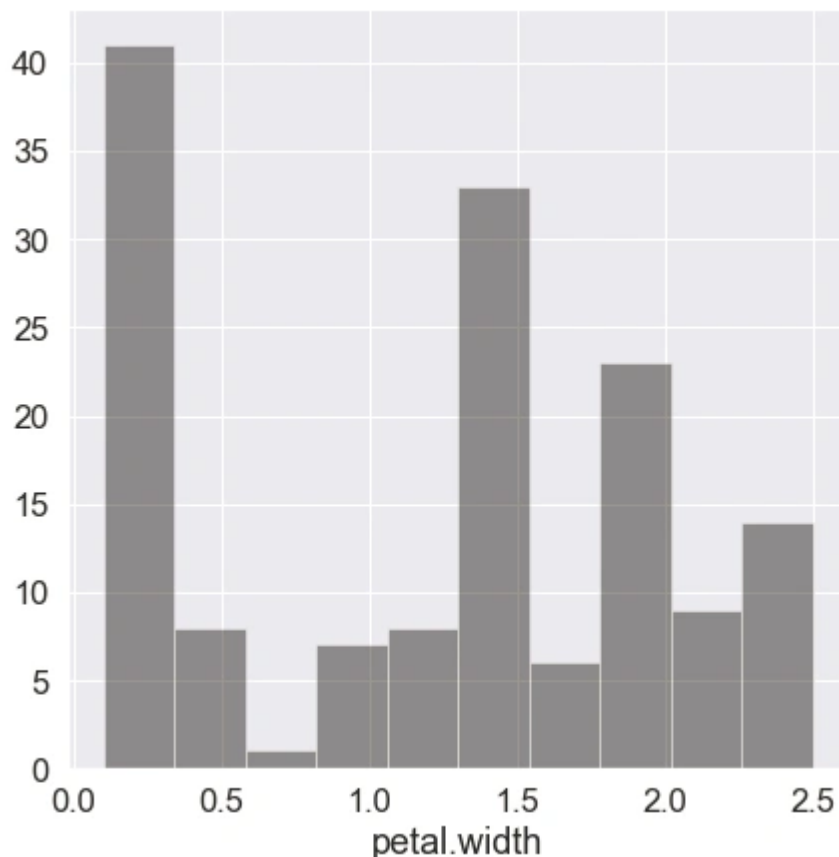
```
In [12]: plt.hist(df['petal.width'])
```

```
Out[12]: (array([41.,  8.,  1.,  7.,  8., 33.,  6., 23.,  9., 14.]),  
          array([0.1 , 0.34, 0.58, 0.82, 1.06, 1.3 , 1.54, 1.78, 2.02, 2.26, 2.5 ]),  
          <a list of 10 Patch objects>)
```



The seaborn function *sns.distplot()* can also be used to plot a histogram.

```
sns.distplot(df['petal.width'], kde=False, color='black', bins=10)
```



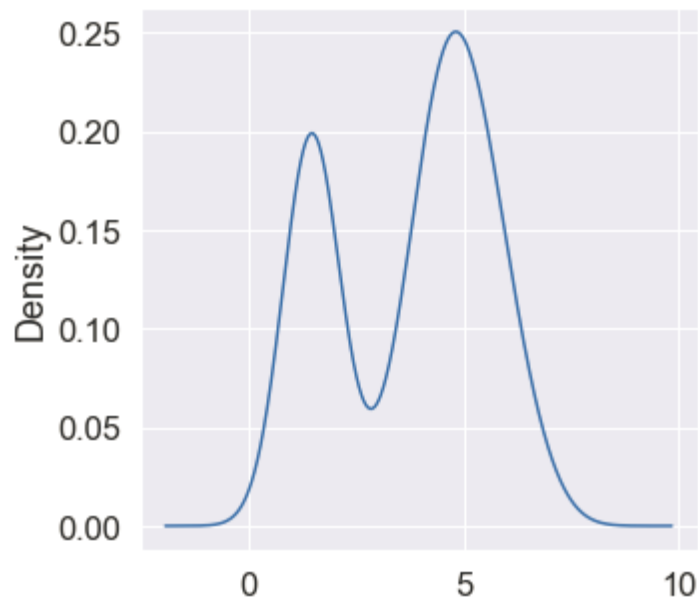
Set the kde (kernel density) parameter to False to display only the histogram. Adjust other parameters like bins (number of bins in the histogram) and color to achieve the desired plot output.

6. DENSITY PLOTS :

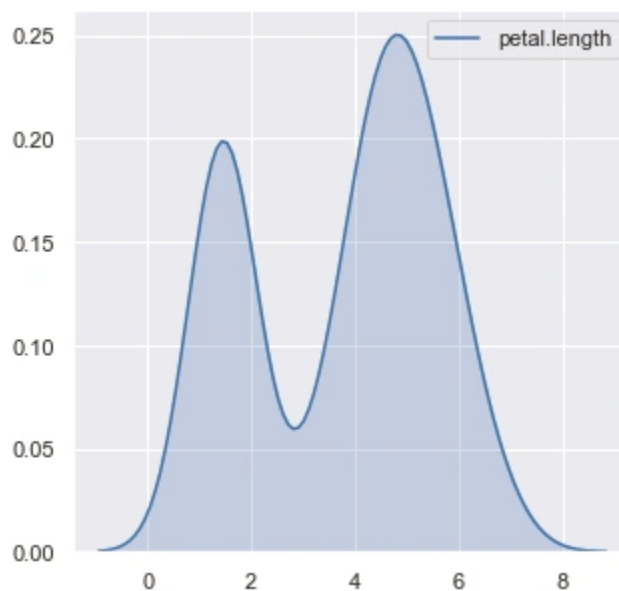
A density plot is like a smoother version of a histogram. In density plots, we use the kernel density estimate to show the probability density function of the variable. The kernel, a continuous curve, is drawn to generate a smooth density estimation for the entire dataset.

Plotting density plot of the variable 'petal.length' :

```
plt.figure(figsize=(5,5))  
df['petal.length'].plot(kind='density')
```



```
sns.set(rc={'figure.figsize': (5, 5)})  
sns.kdeplot(df['petal.length'], shade=True)
```



we use the pandas `df.plot()` function (built over matplotlib) or the seaborn library's `sns.kdeplot()` function to plot a density plot . Many features like shade and type of distribution can be set using the parameters available in the functions. By default, the kernel used is Gaussian (this produces a Gaussian bell curve). Also, other graph smoothing techniques/filters are applicable.

7. RUG PLOTS :

A rug plot is a very simple, but also an ideal legitimate, way of representing a distribution. It consists of vertical lines at each data point. Here, the height is arbitrary. The density of the distribution can be known by how dense the tick-marks are.

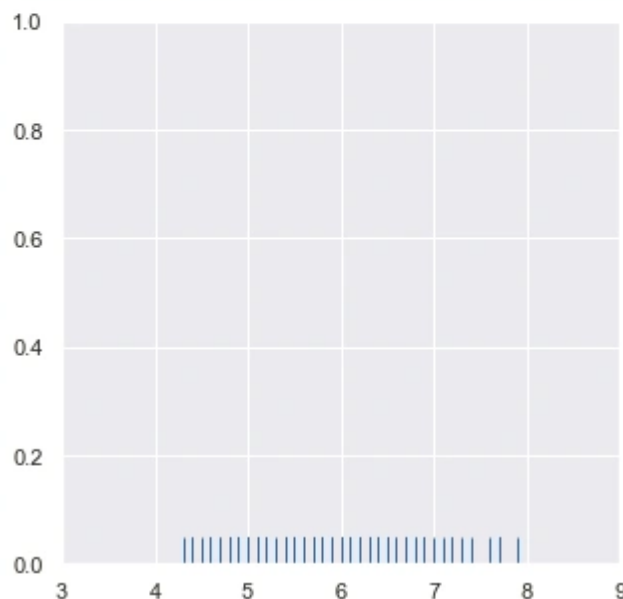
The connection between the rug plot and histogram is very direct: a histogram just creates bins along with the range of the data and then draws a bar with height equal to the number of ticks in each bin. In a rug plot, all of the data points are plotted on a single axis, one tick mark or line for each one.

Compared to a marginal histogram, the rug plot suffers somewhat in terms of readability of the distribution, but it is more compact in its representation of the data. A rug is a very short, long display of point symbols, one for each distinct value. The vertical pipe symbol | often minimizes overlap. While a rug plot may not be a primary plot choice, it can support other plots effectively in specific circumstances.

Plotting the rugs of variable 'sepal .length' :

RUG PLOT

```
fig, ax = plt.subplots()
sns.rugplot(df['sepal.length'])
ax.set_xlim(3,9)
plt.show()
```



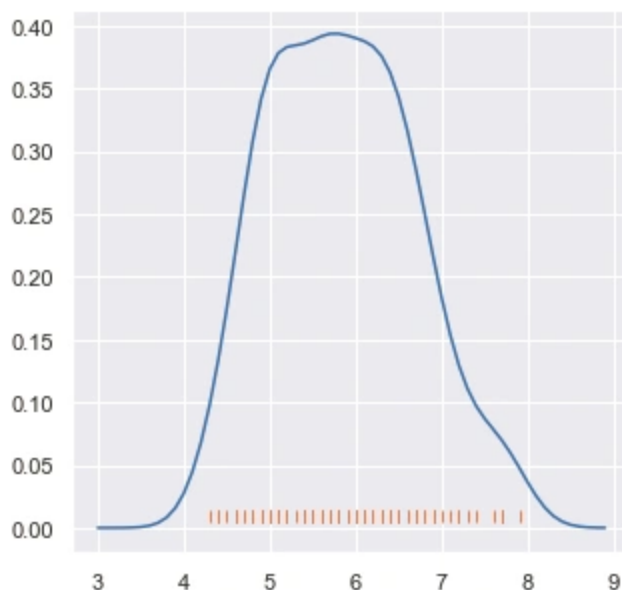


Note :In a few cases, there may be a need to set the range of the values in each axis. In the above illustration, `plt.subplots()` function that returns a figure object and axes object. Using the axes object 'ax' that is passed on to the `set_xlim()` method, the range of values to be considered on the X-axis is set.

Plot a kernel density estimate alongside the rugs to gain a better understanding of the data.

In matplotlib, there is no direct function to create a rug plot. So, we use the `scipy.stats` module to create the required kernel density distribution, and then we plot it using the `plt.plot()` function along with the rugs.

```
from scipy import stats
import numpy as np
kdf=df['sepal.length'].to_numpy()
rdf=np.hstack(kdf)
density = stats.kde.gaussian_kde(rdf)
x = np.arange(3,9,0.1)
plt.plot(x, density(x))
plt.plot(rdf, [0.01]*len(rdf), '|')
```

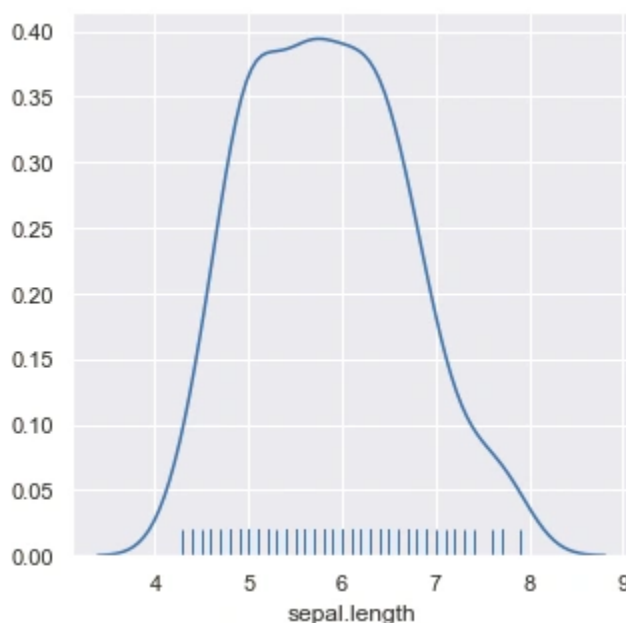


Explanation of the methods used :

The `gaussian_kde()` function generates a kernel-density estimate with Gaussian kernels for univariate data. To prepare the input, convert a dataframe to a numpy array using `to_numpy()` and stack arrays horizontally with `np.hstack()`. This 1-D array, 'rdf', serves as input for `gaussian_kde()`. Define value ranges using `np.arange()` and plot using `plt.plot()` to visualize the results.

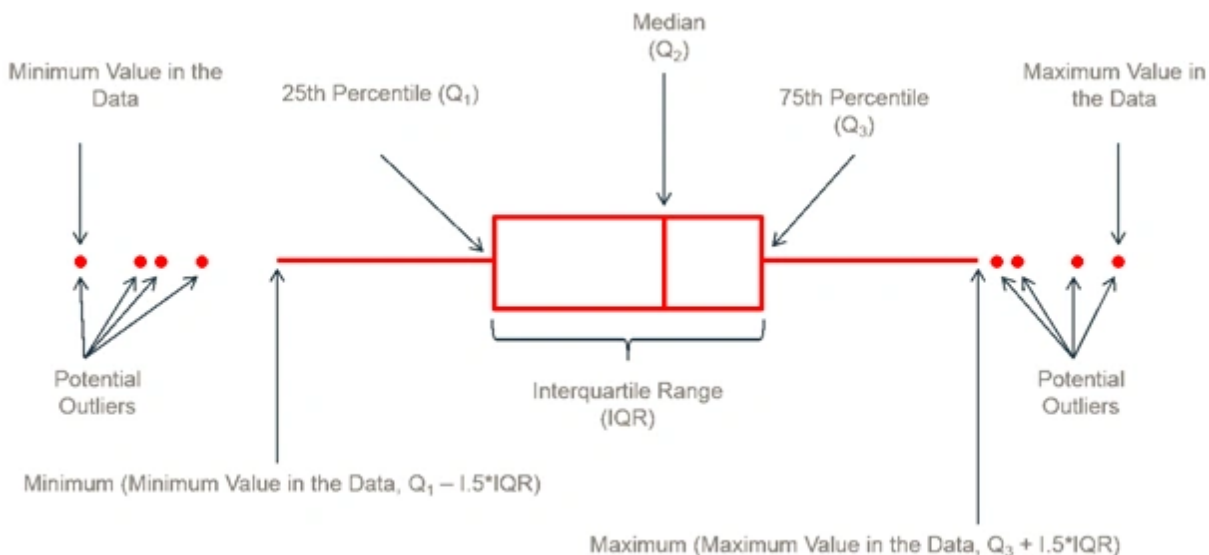
Seaborn library provides a direct and easier function to visualize such a plot with many parameters to play with.

```
sns.distplot(df['sepal.length'], rug=True, hist=False)
```



8. BOX PLOTS :

A box-plot is a very useful and standardized way of displaying the distribution of data based on a five-number summary (minimum, first quartile, second quartile(median), third quartile, maximum). It helps in understanding these parameters of the distribution of data and is extremely helpful in detecting outliers.



(Source:leansigmacorporation.com)

Plotting box plot of variable 'sepal.width' :

```
plt.boxplot(df['sepal.width'])
```

Plotting box plots of all variables in one frame :

Since the box plot is for continuous variables, firstly create a data frame without the column 'variety'. Then drop the column from the DataFrame using the `drop()` function and specify `axis=1` to indicate it.

Removing the column with categorical variables

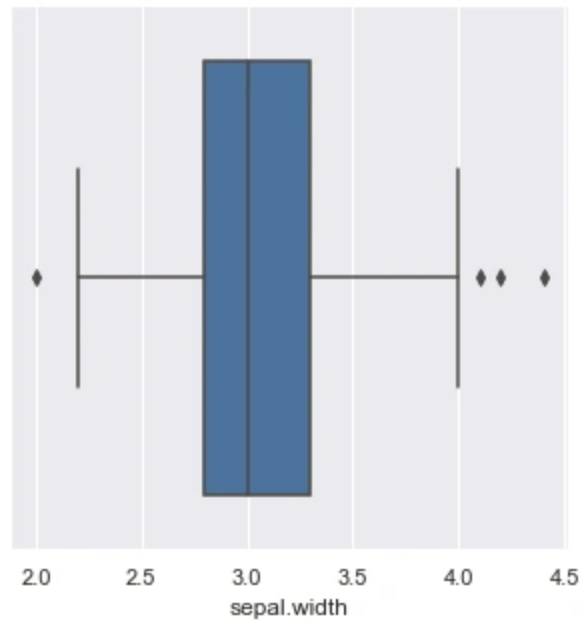
```
dfM=df.drop('variety',axis=1)
```

```
plt.figure(figsize=(9,9))
#Set Title
plt.title('Box plots of the 4 variables')
plt.boxplot(dfM.values,labels=['SepalLength','SepalWidth','PetalLength','PetalWidth'])
```

In matplotlib, mention the labels separately to display it in the output.

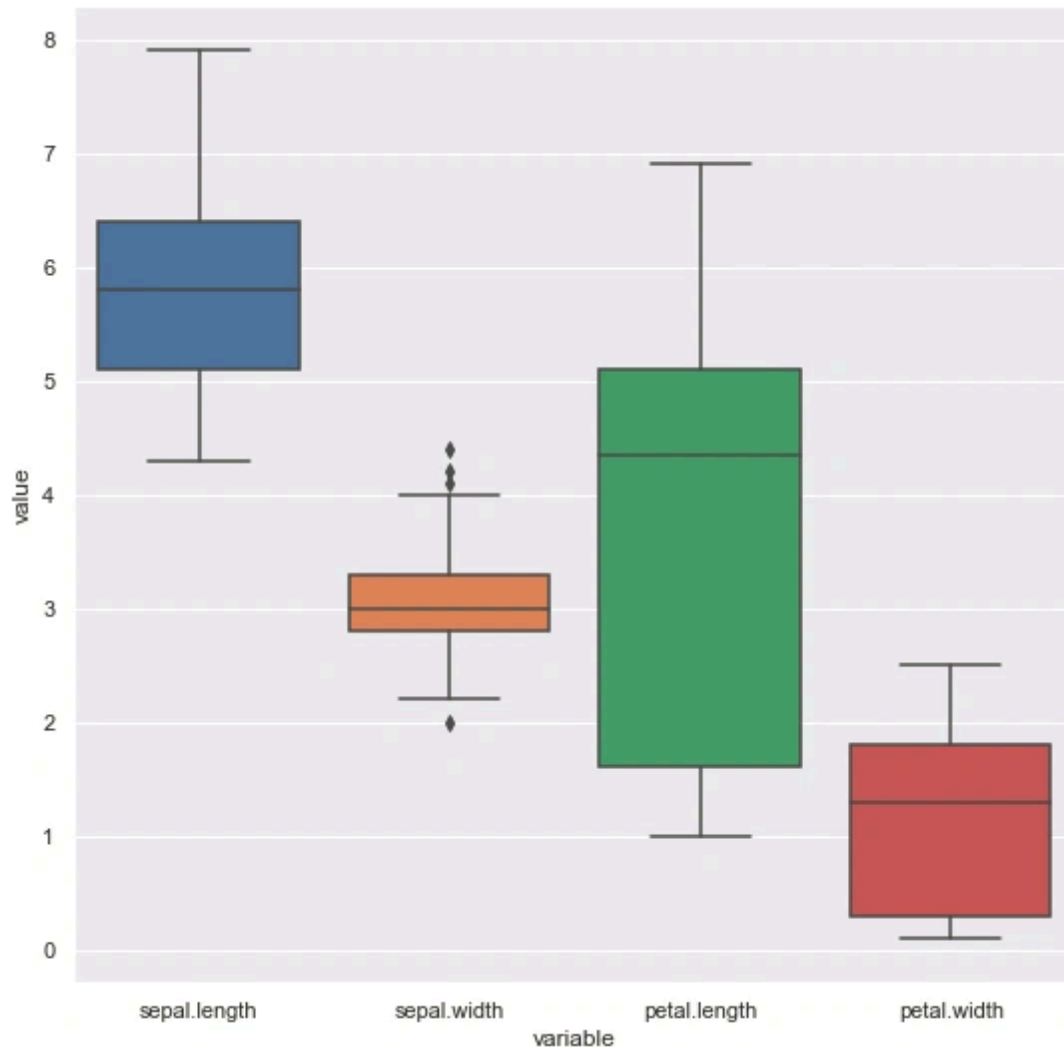
The plotting box plot in seaborn :

```
sns.boxplot(df['sepal.width'])
```



Plotting the box plots of all variables in one frame :

```
sns.set(rc={'figure.figsize': (9,9)})  
sns.boxplot(x="variable", y="value", data=pd.melt(dfM))
```



Pass the modified data frame through the pandas function `pd.melt()` and then to the `sns.boxplot()` function.

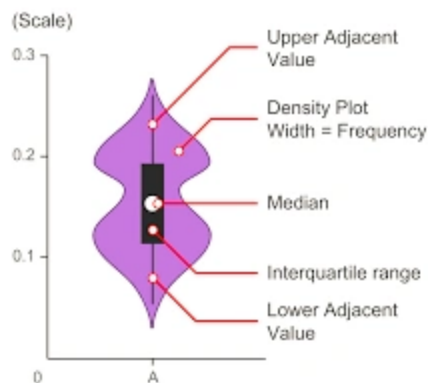
9. `distplot()` :

The `distplot()` function of seaborn library was earlier mentioned under rug plot section. This function combines the matplotlib `hist()` function with the seaborn `kdeplot()` and `rugplot()` functions.

```
sns.set(rc={'figure.figsize': (6,6)})  
sns.distplot(df['petal.length'], color='black', rug=True)
```

10. VIOLIN PLOTS :

The Violin plot is very much similar to a box plot, with the addition of a rotated kernel density plot on each side. This display shows how quantitative data distribute across multiple levels of one (or more) categorical variables, allowing for comparisons between these distributions.



(Source : r-bloggers.com)

```
plt.figure(figsize=(7,7))
plt.violinplot(dfM.values,showmedians=True)
```

We use `plt.violinplot()` function. The Boolean parameter 'showmedians' is set to True, due to which the medians are marked for every variable. The violin plot helps to understand the estimated density of the variable.

In the seaborn library too, the function used to plot a violin plot is similar.

```
sns.set(rc={'figure.figsize':(5,5)})
sns.violinplot(df['sepal.width'],orient='vertical')
```

Comparing the variable 'sepal.width' according to the 'variety' of species mentioned in the dataset :

```
sns.set(rc={'figure.figsize':(9,9)})
sns.violinplot(x=df['variety'], y=df['petal.width'],data=df)
```

VISUALIZING CATEGORICAL VARIABLES :

11. BAR CHART :

The bar plot is a univariate data visualization plot on a two-dimensional axis. One axis is the category axis indicating the category, while the second axis is the value axis that shows the numeric value of that category, indicated by the length of the bar.

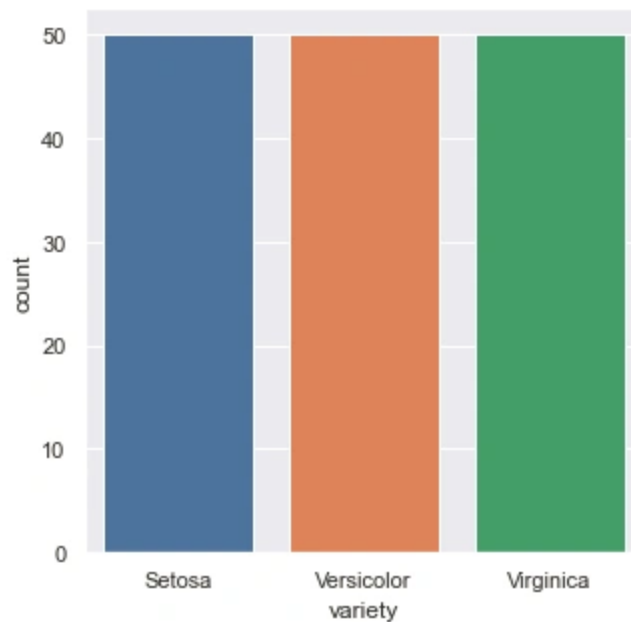
The *plot.bar()* function plots a bar plot of a categorical variable. The *value_counts()* returns a series containing the counts of unique values in the variable.

```
df['variety'].value_counts().plot.bar()
```

The *countplot()* function of the seaborn library obtains a similar bar plot. There is no need to separately calculate the count when using the *sns.countplot()* function.

Since the variety is equally distributed, we obtain bars with equal heights.

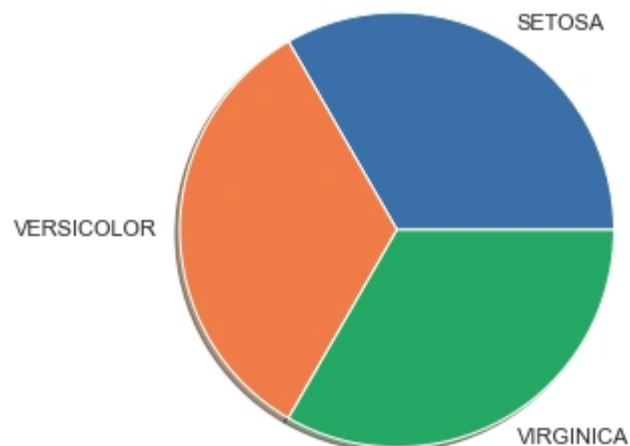
```
sns.countplot(df['variety'])
```



12. PIE CHART :

To visualize the numerical proportion occupied by each category, use the `plt.pie()` function. Divide the sections equally in the pie chart since the categories are equally distributed. Then, pass the array of values to the 'labels' parameter to add labels.

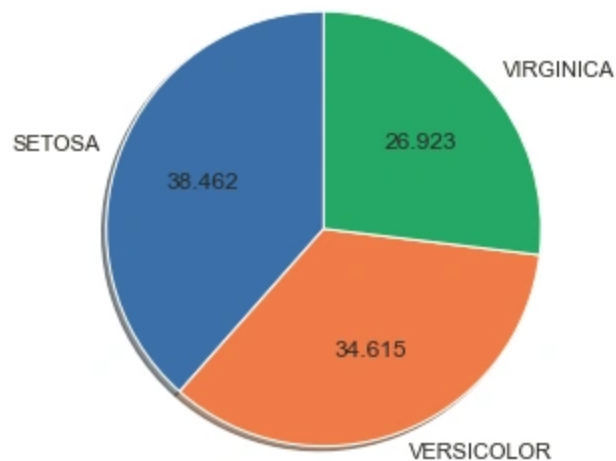
```
plt.pie(df['variety'].value_counts(),  
        labels=['SETOSA', 'VERSICOLOR', 'VIRGINICA'], shadow=True)
```



The 'startangle' parameter of the *pie()* function rotates everything counter-clockwise at a specific angle. Further, the default value for startangle is 0. The 'autopct' parameter enables one to display the percentage value using Python string formatting.

```
df1=df.sample(frac=0.35)
```

```
plt.figure(figsize=(5,5))  
plt.pie(df1['variety'].value_counts(),startangle=90,autopct='%.3f',  
        labels=['SETOSA','VERSICOLOR','VIRGINICA'],shadow=True)
```



Most of the methods that help in the visualization of univariate data have been outlined in this article. As stated before the ability to see the structure and information carried by the data lies in its visual presentation.

Conclusion

Data visualization is a powerful tool for revealing trends, patterns, and insights in data. This article explored various techniques for visualizing univariate analysis in python continuous and categorical data using Python libraries like Matplotlib and Seaborn. We covered enumerative plots such as scatter plots and line plots, as well as summary plots like histograms, density plots, box plots, and violin plots. For categorical data, bar charts and pie charts were discussed. The choice of visualization depends on the data type and analysis goals. Combining multiple visualization techniques can provide a comprehensive view of the data, enabling better understanding and decision-making.

Key Takeaways:

- Data visualization helps reveal trends, patterns, and insights in data
- Different plot types are suitable for different data types and analysis goals
- Python libraries like Matplotlib and Seaborn offer powerful visualization tools
- Combining multiple visualization techniques can provide a comprehensive view of the data

References

1. https://www.rpubs.com/harshaash/Univariate_analysis
2. <https://www.wikipedia.org/> (For basic material on each plot)
3. <https://matplotlib.org/>
4. <https://seaborn.pydata.org/>

Beginner

Data Visualization

Libraries

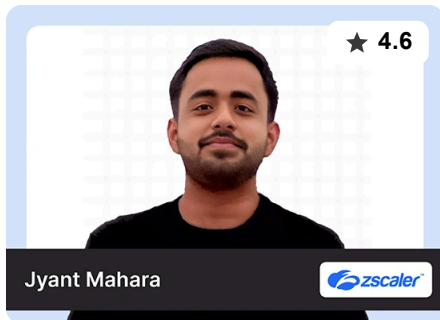
Python

Python

Structured Data

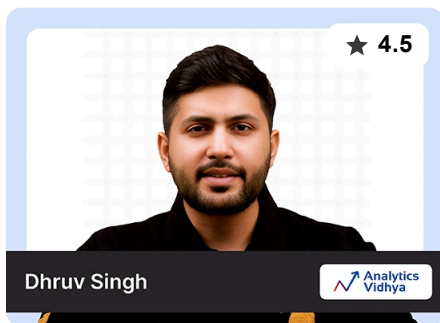
Technique

Free Courses



Exploratory Data Analysis with Python & GenAI

Learn EDA with Python: Transform data into insights using PandasAI & more.



Analyzing Data with Power BI

Turn raw data into insights with Power BI - dashboards, reports & more!

RECOMMENDED ARTICLES

[GPT-4 vs. Llama 3.1 – Which Model is Better?](#)

[Llama-3.1-Storm-8B: The 8B LLM Powerhouse Surpa...](#)

[A Comprehensive Guide to Building Agentic RAG S...](#)

[Top 10 Machine Learning Algorithms in 2025](#)

[45 Questions to Test a Data Scientist on Basics...](#)

[90+ Python Interview Questions and Answers \(202...](#)

[8 Easy Ways to Access ChatGPT for Free](#)

[Prompt Engineering: Definition, Examples, Tips ...](#)

[What is LangChain?](#)

[What is Retrieval-Augmented Generation \(RAG\)?](#)

Responses From Readers

What are your thoughts?...

Submit reply



Zahraa

Thanks so much for this awesome article



Meena

Beautifully and logically well explained all data graphs.
Thanks



Frequently Asked Questions

What is a univariate visualization?

A. A univariate visualization displays and analyzes the distribution of a single variable, helping to understand its characteristics and patterns.

What is an example of univariate data?

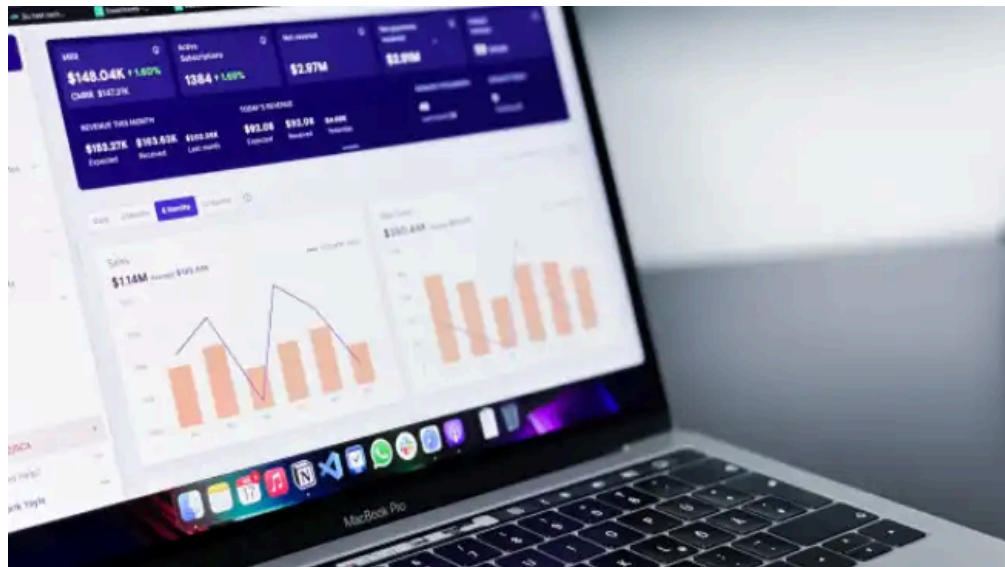
How can univariate data be described and visualized?

What is univariate, bivariate, and multivariate analysis in visualization?

Become an Author →

Share insights, grow your voice, and inspire the data community.

- Reach a Global Audience
- Share Your Expertise with the World
- Build Your Brand & Audience
- Join a Thriving AI Community
- Level Up Your AI Game
- Expand Your Influence in Genrative AI



Flagship Programs

GenAI Pinnacle Program | GenAI Pinnacle Plus Program | AI/ML BlackBelt Program | Agentic AI Pioneer Program

Free Courses

Generative AI | DeepSeek | OpenAI Agent SDK | LLM Applications using Prompt Engineering | DeepSeek from Scratch | Stability.AI | SSM & MAMBA | RAG Systems using LlamaIndex | Building LLMs for Code | Python | Microsoft Excel | Machine Learning | Deep Learning | Mastering Multimodal RAG | Introduction to Transformer Model | Bagging & Boosting | Loan Prediction | Time Series Forecasting | Tableau | Business Analytics | Vibe Coding in Windsurf | Model Deployment using FastAPI | Building Data Analyst AI Agent | Getting started with OpenAI o3-mini | Introduction to Transformers and Attention Mechanisms

Popular Categories

AI Agents | Generative AI | Prompt Engineering | Generative AI Application | News | Technical Guides | AI Tools | Interview Preparation | Research Papers | Success Stories | Quiz | Use Cases | Listicles

Generative AI Tools and Techniques

GANs | VAEs | Transformers | StyleGAN | Pix2Pix | Autoencoders | GPT | BERT | Word2Vec | LSTM | Attention Mechanisms | Diffusion Models | LLMs | SLMs | Encoder Decoder Models | Prompt Engineering | LangChain | LlamaIndex | RAG | Fine-tuning | LangChain AI Agent | Multimodal Models | RNNs | DCGAN | ProGAN | Text-to-Image Models | DDPM | Document Question Answering | Imagen | T5 (Text-to-Text Transfer Transformer) | Seq2seq Models | WaveNet | Attention Is All You Need (Transformer Architecture) | WindSurf | Cursor

Popular GenAI Models

Llama 4 | Llama 3.1 | GPT 4.5 | GPT 4.1 | GPT 4o | o3-mini | Sora | DeepSeek R1 | DeepSeek V3 | Janus Pro | Veo 2 | Gemini 2.5 Pro | Gemini 2.0 | Gemma 3 | Claude Sonnet 3.7 | Claude 3.5 Sonnet | Phi 4 | Phi 3.5 | Mistral Small 3.1 | Mistral NeMo | Mistral-7b | Bedrock | Vertex AI | Qwen QwQ 32B | Qwen 2 | Qwen 2.5 VL | Qwen Chat | Grok 3

AI Development Frameworks

n8n | LangChain | Agent SDK | A2A by Google | SmolAgents | LangGraph | CrewAI | Agno | LangFlow | AutoGen | LlamaIndex | Swarm | AutoGPT

Data Science Tools and Techniques

Python | R | SQL | Jupyter Notebooks | TensorFlow | Scikit-learn | PyTorch | Tableau | Apache Spark | Matplotlib | Seaborn | Pandas | Hadoop | Docker | Git | Keras | Apache Kafka | AWS | NLP | Random Forest | Computer Vision | Data Visualization | Data Exploration | Big Data | Common Machine Learning Algorithms | Machine Learning | Google Data Science Agent

Company

Discover

About Us

Blogs

Contact Us

Expert Sessions

Careers

Learning Paths

Comprehensive Guides

Learn[Free Courses](#)[AI&ML Program](#)[Pinnacle Plus Program](#)[Agentic AI Program](#)**Contribute**[Become an Author](#)[Become a Speaker](#)[Become a Mentor](#)[Become an Instructor](#)**Engage**[Community](#)[Hackathons](#)[Events](#)[Podcasts](#)**Enterprise**[Our Offerings](#)[Trainings](#)[Data Culture](#)[AI Newsletter](#)

[Terms & conditions](#) • [Refund Policy](#) • [Privacy Policy](#) • [Cookies Policy](#) © Analytics Vidhya
2025.All rights reserved.